

ДЕТАЛЬНЫЙ ПЛАН РАЗРАБОТКИ

Электронный учебник «Алгоритмы и структуры данных»

Дипломный проект | Абдулатипова А.З. | ИСПк-402-52-00

ФИНАЛЬНЫЙ ТЕХНОЛОГИЧЕСКИЙ СТЕК

КОМПОНЕНТ	ТЕХНОЛОГИЯ / ВЕРСИЯ	НАЗНАЧЕНИЕ
Клиент	React 18 + TypeScript 5 + Vite 5	SPA, компонентная архитектура
Стилизация	Tailwind CSS 3 + shadcn/ui + Radix UI	UI-компоненты, адаптивная вёрстка
Сервер	Node.js 20 LTS + Express 4	REST API, бизнес-логика, middleware
База данных	PostgreSQL 15	Реляционное хранилище, ACID
ORM	Prisma 5	Type-safe миграции, модели, запросы
Аутентификация	JWT (jsonwebtoken) + bcrypt	Stateless auth, хеширование паролей
ИИ-провайдер 1	OpenAI API (GPT-4o-mini)	Объяснение ошибок, генерация заданий
ИИ-провайдер 2	GigaChat API (Sber)	Резервный провайдер, русскоязычный контекст
ИИ-абстракция	Собственный AI Service Layer	Унифицированный интерфейс, failover
Редактор кода	Monaco Editor (@monaco-editor/react)	Подсветка синтаксиса, IntelliSense
Визуализация	Canvas API + requestAnimationFrame	Пошаговая анимация алгоритмов
Валидация	Zod 3	Runtime type checking, схемы API
Тестирование	Vitest + React Testing Library	Unit-тесты, интеграционные тесты

КОМПОНЕНТ	ТЕХНОЛОГИЯ / ВЕРСИЯ	НАЗНАЧЕНИЕ
Документация API	OpenAPI 3 + Swagger UI	Автоматическая документация endpoints

Архитектура: Трёхуровневая (Presentation → Business Logic → Data). Паттерн: MVC на сервере, Atomic Design на клиенте. Протоколы: HTTP/REST, WebSocket (для real-time прогресса).

0. АРХИТЕКТУРНЫЕ РЕШЕНИЯ И ОГРАНИЧЕНИЯ

0.1. Почему отказались от Supabase

В исходной версии диплома использовался Supabase как BaaS (Backend-as-a-Service) с Edge Functions для ИИ-интеграции. Это решение было заменено на самостоятельный Node.js + Express сервер по следующим причинам:

- Требования задания на ДП: явно указана необходимость выбора серверной платформы (п. 2.3.1) — использование BaaS не демонстрирует компетенции в backend-разработке.
- Трёхуровневая архитектура: Supabase скрывает уровень бизнес-логики, не позволяя продемонстрировать проектирование слоя service/repository.
- ИИ-failover: собственный сервер позволяет реализовать абстрактный AI Service Layer с маршрутизацией между OpenAI и GigaChat на уровне бизнес-логики, а не через костыли на клиенте.
- Контроль производительности: кэширование ответов ИИ, rate limiting, логирование запросов — всё реализуется на уровне Express middleware.
- Дипломная защита: самостоятельно развёрнутый backend демонстрирует глубину проработки проекта.

0.2. Паттерн AI Service Layer (Dual Provider)

Для обеспечения отказоустойчивости и независимости от санкционных ограничений реализован паттерн Strategy с двумя провайдерами ИИ. Интерфейс IAIPProvider абстрагирует конкретную реализацию:

```
interface IAIPProvider {  
  generateExplanation(prompt: AIPrompt): Promise<AIResponse>;  
  generateQuestion(context: QuestionContext): Promise<QuizQuestion>;  
  analyzeCode(code: string, language: string): Promise<CodeAnalysis>;  
  healthCheck(): Promise<boolean>;  
}
```

Реализации: OpenAIProvider (GPT-4o-mini) и GigaChatProvider (Sber GigaChat). Фабрика AIProviderFactory выбирает провайдер по приоритету: OpenAI — primary, GigaChat — fallback при healthCheck() === false или при превышении rate limit. Переключение происходит прозрачно для клиента.

0.3. Безопасность выполнения пользовательского кода

В исходном дипломе использовался new Function() для выполнения кода студента — критическая уязвимость (XSS, доступ к window, document, localStorage). Заменено на:

- Web Workers: код выполняется в изолированном потоке без доступа к DOM.
- Статический анализ: AST-парсинг через @babel/parser — запрещены: eval, Function constructor, fetch, XMLHttpRequest, WebSocket, import/export.
- Таймаут: принудительное прерывание выполнения через Atomics.wait + SharedArrayBuffer после 5 секунд.

- Белый список API: разрешены только: Math, Array, Object, Number, String, console.log (перехватывается в визуализацию).

1. ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ (PostgreSQL + Prisma)

1.1. Доработка схемы относительно исходного диплома

В исходной версии диплома отсутствовала таблица tests для хранения вопросов и вариантов ответов (упоминалась в разделе 2.5, но не была создана в DDL). Добавлена полноценная схема тестирования с поддержкой различных типов вопросов (single choice, multiple choice, matching, short answer).

1.2. Prisma Schema (prisma/schema.prisma)

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url = env("DATABASE_URL")
}

enum UserRole {
  student
  teacher
  admin
}

enum DifficultyLevel {
  easy
  medium
  hard
}

enum QuestionType {
  single_choice
  multiple_choice
  matching
  short_answer
}

model User {
  user_id Int @id @default(autoincrement())
  username String @unique @db.VarChar(100)
  email String @unique @db.VarChar(150)
  password_hash String @db.VarChar(255)
  role UserRole
  created_at DateTime @default(now())

  progress UserProgress[]
  solutions UserSolution[]
  ai_feedbacks AIFeedback[]
}

model Algorithm {
  algorithm_id Int @id @default(autoincrement())
  name String @db.VarChar(150)
  category String @db.VarChar(100)
  difficulty DifficultyLevel
  description String? @db.Text
}
```

```

time_complexity String? @db.VarChar(50)
space_complexity String? @db.VarChar(50)

theory_materials TheoryMaterial[]
tasks Task[]
tests Test[]
}

model TheoryMaterial {
material_id Int @id @default(autoincrement())
algorithm_id Int
title String @db.VarChar(200)
content String @db.Text
type String? @db.VarChar(50)
order_num Int

algorithm Algorithm @relation(fields: [algorithm_id], references: [algorithm_id],
onDelete: Cascade)
tasks Task[]
}

model Test {
test_id Int @id @default(autoincrement())
algorithm_id Int
title String @db.VarChar(200)
description String? @db.Text
passing_score Int @default(70)

algorithm Algorithm @relation(fields: [algorithm_id], references: [algorithm_id],
onDelete: Cascade)
questions Question[]
attempts TestAttempt[]
}

model Question {
question_id Int @id @default(autoincrement())
test_id Int
question_text String @db.Text
question_type QuestionType
explanation String? @db.Text
order_num Int

test Test @relation(fields: [test_id], references: [test_id], onDelete: Cascade)
options Option[]
answers UserAnswer[]
}

model Option {
option_id Int @id @default(autoincrement())
question_id Int
option_text String @db.Text
is_correct Boolean @default(false)
order_num Int

question Question @relation(fields: [question_id], references: [question_id], onDelete:
Cascade)
}

model Task {
task_id Int @id @default(autoincrement())
algorithm_id Int
material_id Int?
name String @db.VarChar(200)
description String @db.Text

```

```

correct_answer String? @db.Text
starter_code String? @db.Text
language String? @db.VarChar(20)

algorithm Algorithm @relation(fields: [algorithm_id], references: [algorithm_id],
onDelete: Cascade)
material TheoryMaterial? @relation(fields: [material_id], references: [material_id],
onDelete: SetNull)
solutions UserSolution[]
}

model UserProgress {
progress_id Int @id @default(autoincrement())
user_id Int
algorithm_id Int
theory_completed Boolean @default(false)
test_completed Boolean @default(false)
practice_completed Boolean @default(false)
score_percent Int?
completed_at DateTime?

user User @relation(fields: [user_id], references: [user_id], onDelete: Cascade)
}

model TestAttempt {
attempt_id Int @id @default(autoincrement())
test_id Int
user_id Int
score Int
max_score Int
passed Boolean
started_at DateTime @default(now())
completed_at DateTime?

test Test @relation(fields: [test_id], references: [test_id], onDelete: Cascade)
answers UserAnswer[]
}

model UserAnswer {
answer_id Int @id @default(autoincrement())
attempt_id Int
question_id Int
answer_text String @db.Text
is_correct Boolean

attempt TestAttempt @relation(fields: [attempt_id], references: [attempt_id], onDelete:
Cascade)
question Question @relation(fields: [question_id], references: [question_id], onDelete:
Cascade)
}

model UserSolution {
solution_id Int @id @default(autoincrement())
user_id Int
task_id Int
code String @db.Text
language String @db.VarChar(20)
result String? @db.VarChar(100)
score Int @default(0)
execution_time Int?
submission_date DateTime @default(now())

user User @relation(fields: [user_id], references: [user_id], onDelete: Cascade)
task Task @relation(fields: [task_id], references: [task_id], onDelete: Cascade)
}

```

```

ai_feedbacks AIFeedback[]
}

model AIFeedback {
  feedback_id Int @id @default(autoincrement())
  solution_id Int?
  user_id Int
  prompt_type String @db.VarChar(50)
  prompt_content String @db.Text
  ai_response String @db.Text
  provider_used String @db.VarChar(50)
  created_at DateTime @default(now())

  solution UserSolution? @relation(fields: [solution_id], references: [solution_id],
  onDelete: SetNull)
  user User @relation(fields: [user_id], references: [user_id], onDelete: Cascade)
}

```

1.3. Миграции и сиды

Миграции: `npx prisma migrate dev --name init` для инициализации, `npx prisma migrate deploy` для production. Сиды: скрипт `prisma/seed.ts` создаёт:

- 4 алгоритма (bubble-sort, insertion-sort, selection-sort, binary-search)
- Теоретические материалы (6 блоков на алгоритм)
- Тесты (5-10 вопросов на алгоритм)
- Практические задания (starter code + эталонное решение)
- Администратор (role: admin) для первичного доступа

2. РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ (Node.js + Express + TypeScript)

2.1. Структура проекта

```
server/
├── src/
│   ├── config/ # Конфигурация (DB, AI, JWT)
│   ├── controllers/ # HTTP-обработчики (thin controllers)
│   ├── services/ # Бизнес-логика (fat services)
│   ├── repositories/ # Доступ к данным (Prisma)
│   ├── middleware/ # Auth, validation, error handling, rate limit
│   ├── routes/ # Маршрутизация
│   ├── types/ # TypeScript interfaces
│   ├── utils/ # Хелперы, validators
│   ├── ai/ # AI Service Layer
│   ├── interfaces/
│   ├── providers/
│   └── factory.ts
├── prisma/
│   ├── schema.prisma
│   └── seed.ts
├── tests/ # Vitest тесты
├── Dockerfile
└── docker-compose.yml
```

2.2. Ключевые endpoints

Метод	Путь	Описание	Auth
POST	/api/auth/register	Регистрация пользователя	Нет
POST	/api/auth/login	Авторизация, выдача JWT	Нет
GET	/api/algorithms	Список алгоритмов с фильтрацией JWT	
GET	/api/algorithms/:id	Детали алгоритма + теория	JWT
GET	/api/algorithms/:id/tests	Тесты алгоритма	JWT
POST	/api/tests/:id/attempt	Начать попытку теста	JWT
POST	/api/tests/attempt/:id/submit	Отправить ответы	JWT
POST	/api/ai/explain	Объяснение ошибки от ИИ	JWT
POST	/api/ai/generate-question	Генерация нового вопроса	JWT
POST	/api/ai/analyze-code	Анализ кода студента	JWT
POST	/api/solutions	Сохранить решение задачи	JWT
GET	/api/progress	Прогресс пользователя	JWT
GET	/api/admin/users	Список пользователей	Admin JWT

Метод	Путь	Описание	Auth
POST	/api/admin/algorithms	Создать алгоритм	Admin JWT
PUT	/api/admin/algorithms/:id	Обновить алгоритм	Admin JWT

2.3. AI Service Layer — детальная реализация

Файл `src/ai/factory.ts` реализует логику выбора провайдера с учётом health-check и circuit breaker pattern:

```
class AIProviderFactory {
  private providers: IAIPProvider[] = [
    new OpenAIProvider(),
    new GigaChatProvider()
  ];
  private circuitBreaker = new Map();
  private readonly CIRCUIT_THRESHOLD = 3;
  private readonly CIRCUIT_TIMEOUT = 60000; // 1 min

  async getProvider(): Promise {
    for (const provider of this.providers) {
      const name = provider.constructor.name;
      const state = this.circuitBreaker.get(name);

      if (state && state.failures >= this.CIRCUIT_THRESHOLD) {
        if (Date.now() - state.lastFailure < this.CIRCUIT_TIMEOUT) {
          continue; // Circuit open
        }
        this.circuitBreaker.set(name, { failures: 0, lastFailure: 0 });
      }

      const healthy = await provider.healthCheck().catch(() => false);
      if (healthy) return provider;

      this.recordFailure(name);
    }
    throw new AIProviderUnavailableError('All providers failed');
  }

  private recordFailure(name: string): void {
    const state = this.circuitBreaker.get(name) || { failures: 0, lastFailure: 0 };
    state.failures++;
    state.lastFailure = Date.now();
    this.circuitBreaker.set(name, state);
  }
}
```

2.4. Rate Limiting и кэширование

Rate Limiting: `express-rate-limit` — 100 запросов/15 мин на IP, 20 запросов/мин на `/api/ai/*` endpoints. Кэширование ИИ-ответов: in-memory LRU cache (max 1000 entries, TTL 1 час) для идентичных промптов. Ключ кэша — SHA256 хеш нормализованного промпта. Снижает затраты на API на ~40%.

3. РАЗРАБОТКА КЛИЕНТСКОЙ ЧАСТИ (React + TypeScript + Vite)

3.1. Структура проекта (Atomic Design)

```
client/
├── src/
│   ├── components/ # Atomic Design
│   │   ├── atoms/ # Button, Input, Badge, Icon
│   │   ├── molecules/ # AlgorithmCard, StatTile, QuestionOption
│   │   ├── organisms/ # TheoryBlock, VisualizationCanvas, CodePanel
│   │   ├── templates/ # AlgorithmLayout, AdminLayout
│   │   └── pages/ # CatalogPage, AlgorithmPage, AdminPage
│   ├── hooks/ # useAlgorithm, useVisualization, useAI
│   ├── stores/ # Zustand state management
│   ├── services/ # API clients (axios instances)
│   ├── types/ # Shared TypeScript types
│   ├── utils/ # Algorithm generators, validators
│   ├── workers/ # Web Workers для выполнения кода
│   └── styles/ # Tailwind config, globals
├── public/
├── index.html
├── vite.config.ts
├── tailwind.config.js
└── tsconfig.json
```

3.2. State Management (Zustand)

Отказ от Redux в пользу Zustand — минимальный boilerplate, отличная TypeScript поддержка, нет необходимости в Provider-обёртках. Структура store:

```
// stores/algorithmStore.ts
interface AlgorithmState {
  algorithms: Algorithm[];
  currentAlgorithm: Algorithm | null;
  currentStep: number;
  steps: AlgorithmStep[];
  isPlaying: boolean;
  speed: number; // ms between steps
}

// Actions
loadAlgorithms: () => Promise;
selectAlgorithm: (id: string) => void;
generateSteps: (code: string, input: number[]) => void;
play: () => void;
pause: () => void;
stepForward: () => void;
stepBackward: () => void;
setSpeed: (speed: number) => void;
reset: () => void;
}

export const useAlgorithmStore = create((set, get) => ({
  // ... implementation
}));
```

3.3. Модуль визуализации алгоритмов

Реализован на Canvas API (не SVG) для достижения 60 FPS при анимации массивов до 100 элементов. Архитектура модуля:

- **AlgorithmEngine** — абстрактный класс, от которого наследуются конкретные реализации (**BubbleSortEngine**, **InsertionSortEngine** и т.д.)
- **StepGenerator** — функция, принимающая входной массив и возвращающая массив шагов **AlgorithmStep[]**
- **CanvasRenderer** — отвечает за отрисовку: столбцы, подсветка, стрелки, текстовые аннотации
- **AnimationController** — управляет playback: **requestAnimationFrame**, **scrubbing**, **pause/resume**
- **StatsCollector** — агрегирует **comparisons**, **swaps**, **time elapsed**

3.4. Web Worker для безопасного выполнения кода

```
// workers/code-executor.worker.ts
self.onmessage = (e: MessageEvent) => {
  const { code, array, target } = e.data;
  const steps: AlgorithmStep[] = [];

  const visualize = (
    arr: number[], comparing: number[], sorted: number[],
    description?: string, action: StepAction = "none"
  ) => {
    steps.push({ array: [...arr], comparing, sorted, description, action });
    if (steps.length > 5000) throw new Error("Step limit exceeded");
  };

  try {
    // AST validation before execution
    const forbidden = ['eval', 'Function', 'fetch', 'XMLHttpRequest',
      'WebSocket', 'import', 'export'];
    // ... validation logic

    const func = new Function("arr", "visualize", "target", code);
    func([...array], visualize, target);

    self.postMessage({ type: 'success', steps });
  } catch (err) {
    self.postMessage({ type: 'error', message: err.message });
  }
};
```

3.5. Интеграция с сервером

Axios instance с interceptor-ами:

- **Request interceptor**: добавляет **Authorization: Bearer <token>** из **localStorage**.
- **Response interceptor**: при 401 — редирект на **/login**, при 429 — **exponential backoff retry** (max 3).
- **Base URL**: через **env VITE_API_URL**.

4. ИНТЕГРАЦИЯ ИИ-МОДУЛЯ (Dual Provider)

4.1. OpenAI Provider (GPT-4o-mini)

Модель GPT-4o-mini (а не GPT-4o, как в исходном дипломе) — соответствует заданию на ДП, стоимость ~\$0.15/1M input tokens, ~\$0.60/1M output tokens. Промпт-инжиниринг:

```
// Промпт для объяснения ошибки в тесте
const explainPrompt = {
  model: "gpt-4o-mini",
  messages: [
    {
      role: "system",
      content: `Ты — педагог по алгоритмам и структурам данных.
      Объясняй ошибки студента простым языком, максимум 150 слов.
      Указывай конкретную ошибку и давай рекомендацию по исправлению.`
    },
    {
      role: "user",
      content: `Тема: ${topic}
      Вопрос: ${question}
      Ответ студента: ${userAnswer}
      Правильный ответ: ${correctAnswer}
      Объясни ошибку и предложи новый вопрос для проверки.`
    }
  ],
  temperature: 0.7,
  max_tokens: 300
};
```

4.2. GigaChat Provider (Sber)

GigaChat API — резервный провайдер. Особенности интеграции:

- Аутентификация через OAuth 2.0 (client_credentials): получение access_token с TTL 30 мин.
- Эндпоинт: POST <https://gigachat.devices.sberbank.ru/api/v1/chat/completions>
- Модели: GigaChat:latest (основная), GigaChat-Pro (для сложных задач).
- Промпты адаптированы под русскоязычный контекст — более естественные объяснения для русскоязычных студентов.

4.3. Промпт для анализа кода

```
const codeAnalysisPrompt = {
  model: "gpt-4o-mini",
  messages: [
    {
      role: "system",
      content: `Ты — эксперт по алгоритмам. Проанализируй код студента.
      Верни JSON с полями:
      - issues: массив объектов {line, severity, message, suggestion}
      - complexity: оценка сложности O-нотация
      - improvements: массив рекомендаций по оптимизации`
    },
    {
      role: "user",
      content: `Алгоритм: ${algorithmName}
      Язык: ${language}
      Код студента:
```

```
${studentCode}
```

Эталонное решение:

```
${referenceCode}`  
}  
],  
response_format: { type: "json_object" },  
temperature: 0.3  
};
```

4.4. Кэширование и оптимизация затрат

LRU Cache (библиотека `lru-cache`) на сервере: • Ключ: `SHA256(provider + prompt_hash + temperature)` • TTL: 3600 секунд • Max entries: 1000 • Хит-рейт ожидается ~35-40% для повторяющихся ошибок студентов. Оценка затрат (500 пользователей/мес, ~20 ИИ-запросов на пользователя): • Без кэша: ~\$30-50/мес (OpenAI) • С кэшем: ~\$18-30/мес • GigaChat: бесплатный лимит 1M tokens/мес, далее ~₽300/мес

5. РЕАЛИЗАЦИЯ КЛЮЧЕВЫХ ЭКРАННЫХ ФОРМ

5.1. Экран каталога алгоритмов (/algorithms)

Компонент `CatalogPage` — точка входа пользователя. Функционал:

- Фильтрация: по категории (сортировка, поиск, графы), сложности (easy/medium/hard), статусу прохождения.
- Поиск: `debounced input` (`lodash.debounce, 300ms`), fuzzy matching через `fuse.js`.
- Сортировка: по названию, сложности, дате добавления.
- Адаптивная сетка: CSS Grid с `auto-fill minmax(280px, 1fr)` — от 1 до 4 колонок.
- Lazy loading: бесконечный скролл или пагинация (`page size = 12`).

5.2. Экран изучения теории (/algorithm/:id/theory)

Компонент `TheoryPage`. Ключевые особенности:

- Блокировка прогресса: следующий блок разблокируется только после правильного ответа на контрольный вопрос текущего блока.
- Контрольные вопросы: `inline-quiz` после каждого блока. При ошибке — запрос к `/api/ai/explain`, отображение объяснения + новый вопрос.
- Прогресс-бар: визуальный индикатор прохождения (`completed blocks / total blocks`).
- Заметки: локальное хранилище (`localStorage`) для пользовательских заметок по блоку.
- Переход к практике: по завершении всех блоков — кнопка «Перейти к визуализации» с `confetti`-анимацией.

5.3. Экран визуализации (/algorithm/:id/visualize)

Компонент `VisualizationPage` — наиболее сложный с технической точки зрения.

- Canvas-визуализация: столбчатая диаграмма, высота пропорциональна значению. Цветовая кодировка: сравниваемые — оранжевый, отсортированные — зелёный, активный — красный.
- Панель управления: Play/Pause, Step Forward/Backward, Reset, скорость (0.5x — 4x).
- Настройка данных: `preset arrays` (random, sorted, reversed, nearly sorted), ручной ввод через `comma-separated input`.
- Дашборд статистики: `comparisons`, `swaps`, `time elapsed`, `memory usage (estimated)`.
- Редактор кода: Monaco Editor в нижней панели (`drag-to-resize`). Автозапуск через `800ms debounce`.
- Сравнение с эталоном: `split-view` — код студента | эталонный код с `diff`-подсветкой.

5.4. Экран тестирования (/algorithm/:id/test)

Компонент TestPage. Логика:

- Типы вопросов: single choice (radio), multiple choice (checkbox), matching (drag-and-drop), short answer (text input).
- Адаптивность: при результате $< 70\%$ — автоматический запрос к ИИ для генерации дополнительных вопросов по слабым темам.
- Таймер: необязательный, отображает время прохождения.
- Результат: процент, разбор ошибок (с ИИ-объяснением), сравнение с предыдущими попытками.
- Сохранение: результат фиксируется в TestAttempt, обновляется UserProgress.

5.5. Административная панель (/admin)

Компонент AdminPage. Доступ по role === 'admin'. Функционал:

- CRUD алгоритмов: создание, редактирование, удаление. WYSIWYG-редактор для теории (@tiptap/react).
- Управление тестами: конструктор вопросов с drag-and-drop для порядка, preview режим.
- Управление пользователями: таблица с фильтрами, изменение ролей, блокировка.
- Аналитика: графики прохождения (Recharts), heatmap активности, экспорт в CSV/PDF.

6. ТЕСТИРОВАНИЕ И АПРОБАЦИЯ

6.1. Unit-тестирование (Vitest)

Покрытие критичных модулей:

- AlgorithmEngine: корректность генерации шагов для всех 4 алгоритмов на фиксированных входных данных.
- AIProviderFactory: circuit breaker, fallback logic, health-check mocking.
- Auth middleware: валидация JWT, проверка ролей, обработка expired tokens.
- Prisma queries: интеграционные тесты с testcontainers (PostgreSQL в Docker).

6.2. E2E тестирование (Playwright)

Сценарии:

- Полный учебный цикл: выбор алгоритма → теория → визуализация → редактор → тест → результат.
- Ошибка в коде → запрос к ИИ → отображение объяснения → исправление → повторный запуск.
- Администратор: создание алгоритма → добавление теории → добавление теста → проверка на клиенте.
- Failover ИИ: блокировка OpenAI → автоматический переход на GigaChat → корректность ответа.

6.3. Нагрузочное тестирование

Инструмент: k6 или Artillery. Сценарий: 100 одновременных пользователей (требование T3), ramp-up 60 секунд, duration 5 минут. Метрики: p95 latency < 2s, error rate < 1%, RPS > 50.

6.4. Экспериментальная апробация

Сценарий апробации (полный цикл «Сортировка пузырьком»):

- Шаг 1 — Выбор: пользователь открывает каталог, фильтрует по «Сортировка», выбирает «Сортировка пузырьком».
- Шаг 2 — Теория: последовательное прохождение 6 блоков. На блоке 3 — намеренная ошибка в тесте, получение ИИ-объяснения, повторный ответ.
- Шаг 3 — Визуализация: запуск с массивом [64, 34, 25, 12, 22, 11, 90]. Пауза на шаге 5, ручной переход назад/вперёд, изменение скорости.
- Шаг 4 — Редактор: намеренная ошибка в условии ($arr[j] < arr[j+1]$ вместо $>$). Автозапуск показывает некорректный результат. Запрос ИИ-анализа.
- Шаг 5 — Тест: прохождение 5 вопросов. Ошибка на вопросе о сложности → ИИ-объяснение → дополнительный вопрос → успешное завершение.
- Шаг 6 — Проверка прогресса: в профиле отображается 100% прохождение алгоритма, график попыток.

7. РАЗВЁРТЫВАНИЕ И DEVOPS

7.1. Docker-контейнеризация

Клиент: многоступенчатая сборка (multi-stage Dockerfile) — этап build на node:20-alpine, этап serve на nginx:alpine со статикой. Размер образа ~25MB. Сервер: node:20-alpine + dumb-init для корректной обработки сигналов. База данных: официальный образ postgres:15-alpine с volume для persistence.

7.2. docker-compose.yml

```
version: '3.8'
services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: algorithms_db
    volumes:
      - pgdata:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  server:
    build: ./server
    environment:
      DATABASE_URL: postgresql://${DB_USER}:${DB_PASSWORD}@db:5432/algorithms_db
      OPENAI_API_KEY: ${OPENAI_API_KEY}
      GIGACHAT_CLIENT_ID: ${GIGACHAT_CLIENT_ID}
      GIGACHAT_CLIENT_SECRET: ${GIGACHAT_CLIENT_SECRET}
      JWT_SECRET: ${JWT_SECRET}
    ports:
      - "3001:3001"
    depends_on:
      - db

  client:
    build: ./client
    ports:
      - "80:80"
    depends_on:
      - server

volumes:
  pgdata:
```

7.3. CI/CD (GitHub Actions)

Pipeline: 1. Lint + Type Check: ESLint, Prettier, tsc --noEmit 2. Unit Tests: Vitest с coverage threshold (min 70%) 3. Build: сборка клиента и сервера 4. E2E Tests: Playwright в Docker-контейнере 5. Deploy: push образов на Docker Hub, деплой на VPS через ssh

8. КАЛЕНДАРНЫЙ ПЛАН (с учётом сроков задания на ДП)

Сроки из задания на ДП: оформление и доработка — 03.06–07.06, представление руководителю — 08.06–10.06, устранение замечаний — 11.06–14.06, получение отзыва — 15.06–19.06, сдача — 20.06.2026.

Этап	Задачи	Срок	Результат
0. Подготовка	Окружение, репозиторий, API-ключи	1 день	Рабочее окружение
1. База данных	Prisma schema, миграции, сиды	2 дня	PostgreSQL с тестовыми данными
2. Backend	Express API, auth, CRUD, AI layer	5 дней	REST API + документация Swagger
3. Frontend core	Vite + React + Tailwind + routing	2 дня	Базовый скелет приложения
4. Каталог + Теория	AlgorithmPage, TheoryPage, блокировка процесса	3 дня	Полноценный учебный модуль
5. Визуализация	Canvas renderer, 4 алгоритма, Web Worker	4 дня	Интерактивная визуализация
6. Редактор + ИИ	Monaco Editor, AI integration, failover	3 дня	Редактор с ИИ-анализом
7. Тестирование модуля	QuizPage, разные типы вопросов, адаптивизация	2 дня	Система тестирования
8. Admin Panel	CRUD, аналитика, управление пользователями	2 дня	Административная панель
9. Тестирование	Unit, E2E, нагрузочное, апробация	3 дня	Тестовый протокол, отчёт
10. Доработка диплома	Исправления, скриншоты, диаграммы	2 дня	Готовая пояснительная записка

Итого: ~29 дней чистой разработки. С учётом параллельной работы над дипломом — выполнимо в срок.

9. ЧЕК-ЛИСТ СООТВЕТСТВИЯ: ЗАДАНИЕ НА ДП ↔ ДИПЛОМ ↔ РЕАЛИЗАЦИЯ

Таблица ниже сводит воедино требования задания на дипломный проект, содержание исходной пояснительной записки и планируемую реализацию. Ячейки с расхождениями выделены.

Пункт задания	Требование	В исходном дипломе	Планируемая реализация	Статус
2.1.1	Анализ предметной области	Есть (раздел 1.1)	Используется как есть, дополнить актуальными данными	
2.1.2	Обзор существующих аналогов	Есть (раздел 1.2, 5 аналогов)	Используется как есть, добавить сравнительную таблицу	
2.1.3	Анализ проблемной области	Есть (раздел 1.3)	Используется как есть	✓
2.1.4	Обзор средств проектирования	Есть (раздел 1.4, Figma/Draw.io)	Используется как есть	✓
2.2.1	Диаграммы функционирования	Есть (IDEF0, DFD, UML)	Все диаграммы должны быть сгенерированы и встроены	
2.2.2	Функциональная диаграмма (IDEF0)	Есть (рис. 2.2, 2.3)	Перегенерировать в draw.io, экспортировать в PNG	
2.2.3	Модель поведения системы	Есть (Use-Case, Sequence)	Перегенерировать в StarUML, обновить акторов	
2.2.4	DFD диаграммы	Есть (рис. 2.6, 2.7)	Адаптировать под новую архитектуру (без Supabase)	
2.2.5	Алгоритмы функционирования	Есть (рис. 2.9–2.12)	Обновить блок-схемы с учётом Node.js backend	
2.2.6	Проектирование модуля ИИ-поддержки	Есть (рис. 2.7, не 2.2.6)	Переименовать 2.7 → 2.2.6, дополнить dual provider	
2.3.1	Выбор серверной платформы	Описан, но НЕ реализован (Supabase)	Node.js 20 + Express + TypeScript	✗→✓
2.3.2	Выбор языка и фреймворка клиента	React + TS + Vite	React 18 + TypeScript 5 + Vite 5	✓
2.3.3	Выбор СУБД	PostgreSQL (через Supabase)	PostgreSQL 15 напрямую + Prisma ORM	
2.3.4	Выбор базовой модели ИИ	GPT-4o (не соответствует заданию)	GPT-4o-mini + GigaChat (dual provider)	
2.3.5	Интеграция ИИ-сервиса и резервный fallback	Есть fallback на YandexGPT/GigaChat	Реализован AI Service Layer с circuit breaker	
2.3.6	Разработка интерфейса	Есть (раздел 3.3, скриншоты)	Полная реализация всех экранных форм	
2.4	Разработка отдельных программных модулей	Не хватает (отсутствует)	Добавить раздел 2.4 с описанием 5 модулей	
2.4.1	Тестирование и экспериментальная апробация	Есть (раздел 3.6)	Расширить: unit + E2E + нагрузочное	
2.4.2	Тестирование отдельных фрагментов	Есть (ручное + автоматическое)	Добавить Vitest + Playwright	✓
2.4.3	Экспериментальная апробация	Есть (сценарий сортировки пугал)	Выполнить полный цикл, зафиксировать скриншоты	
3.1	Демонстрационный материал	Презентация	Презентация + live-demo на защите	

Легенда: ✓ — соответствует заданию, ⚠ — требует доработки/уточнения, ✗→✓ — критическое расхождение, требует исправления.

10. ПРИЛОЖЕНИЕ: ИСПРАВЛЕНИЯ В ДИПЛОМНОЙ РАБОТЕ

10.1. Структурные изменения

Действие	Где	Причина
Поменять местами названия баз данных	Глава 3.1.1 и 3.1.2	В задании 3.1.1 — серверная платформа, 3.1.2 — клиентская
Переименовать раздел 2.7 → 2.2.6	Глава 2	В задании требуется подпункт 2.2.6, в дипломе он отсутствовал
Добавить раздел 2.4	Между 2.3 и 2.5	В задании требуется раздел «Разработка отдельных про...
Исправить GPT-4o → GPT-4o-mini	Везде в тексте	В задании указана именно модель GPT-4o-mini.
Убрать Supabase, добавить Node.js, Express	Раздел 3.1.1, 3.5	Соответствие требованию выбора серверной платформы
Добавить таблицу tests в БД	Раздел 2.5, DDL	В дипломе упоминается, но не реализована.
Заменить new Function() на window.eval()	Глава 3.5.3	Критическая XSS-уязвимость.

10.2. Контентные изменения

Действие	Раздел	Детали
Вставить реальные диаграммы	2.1–2.2.5	IDEF0 (контекстная + декомпозиция), Use-Case, Sequence
Добавить ER-диаграмму	2.5	Сгенерировать из Prisma schema через prisma-erd-generator
Добавить скриншоты интерфейса	3.6.3	Все экранные формы: каталог, теория, визуализация, ред...
Дополнить раздел ИИ-модуля	2.2.6 (ex-2.7)	Описать dual provider, circuit breaker, кэширование, оценк...
Добавить Swagger/OpenAPI	3.1	Автоматическая документация API как приложение к дип...
Добавить тест-кейсы	3.6.1	Формализованные тест-кейсы в табличном виде (прилож...
Обновить список источников	Список литературы	Добавить документацию Node.js, Express, Prisma, Zod, Vit...

10.3. Технические замечания

1. Несоответствие архитектурных диаграмм: рисунок 2.1 подписан как «Трёхуровневая архитектура», но на самом деле изображена двухуровневая. Необходимо перерисовать. 2. Неточности в математическом аппарате (раздел 2.6): • Формула $U_{max} \geq 100$ — не формула, а требование. Заменить на расчёт нагрузки: $RPS = N_{users} \times requests_per_min / 60$, проверка через Little's Law. • Добавить формулу кэш-хитрейта: $HitRate = CacheHits / (CacheHits + CacheMisses) \times 100\%$. 3. Ошибка в разделе 3.1.1: «Процесс разработки пользовательского интерфейса требовал выбора инструмента...» — это текст про клиентскую часть, но раздел называется «Выбор серверной платформы». Полная перепутанность. 4. Неполнота реализации: в дипломе заявлено 4 алгоритма визуализации, но показаны скриншоты только для сортировки пузырьком. Необходимо продемонстрировать все 4 алгоритма. 5. Отсутствие

админ-панели в скриншотах: в ТЗ и дипломе упоминается, но визуального подтверждения нет.

11. ИТОГОВЫЙ ЧЕК-ЛИСТ «ОТ И ДО»

Финальный контрольный список для проверки готовности проекта к защите. Все пункты должны быть отмечены перед сдачей.

№	Категория	Проверяемый элемент	Критичность
1	Инфраструктура	Репозиторий на GitHub с историей коммитов	Обязательно
2	Инфраструктура	Docker + docker-compose поднимаются без ошибок (docker-compose up)	Обязательно
3	База данных	Prisma migrate deploy выполняется успешно	Обязательно
4	База данных	Seed-скрипт создаёт 4 алгоритма + теорию + тесты	Обязательно
5	Backend	Все 15 endpoints из таблицы 2.2 отвечают корректно	Обязательно
6	Backend	JWT-аутентификация работает (регистрация, логин, защита маршрутов)	Обязательно
7	Backend	AI Service Layer переключается между OpenAI и GigaChat при сбое	Обязательно
8	Backend	Rate limiting активен (проверить через 21+ запросов)	Обязательно
9	Backend	Swagger UI доступен по /api/docs	Желательно
10	Frontend	Каталог алгоритмов отображается, фильтрация работает	Обязательно
11	Frontend	Теория: блокировка прогресса, ИИ-объяснение при ошибке	Обязательно
12	Frontend	Визуализация: 4 алгоритма, play/pause/step, настройка данных	Обязательно
13	Frontend	Редактор кода: Мопасо, подсветка, автозапуск, ИИ-анализ	Обязательно
14	Frontend	Тестирование: 5-10 вопросов, адаптивность при <70%	Обязательно
15	Frontend	Адаптивность: корректное отображение на 320px, 768px, 1024px	Обязательно
16	Тестирование	7 тест-кейсов «чёрного ящика» пройдены и задокументированы	Обязательно
17	Тестирование	Unit-тесты: покрытие $\geq 70\%$ критичных модулей	Желательно
18	Тестирование	E2E: полный учебный цикл пройден автоматически	Желательно
19	Тестирование	Нагрузочное: 100 concurrent users, p95 < 2s	Обязательно
20	Апробация	Сценарий «Сортировка пузырьком» пройден полностью, скрипты сохранены	Обязательно
21	Диплом	Все диаграммы вставлены в текст (IDEF0, DFD, UML, ER)	Обязательно
22	Диплом	Все скриншоты интерфейса актуальны и соответствуют реализации	Обязательно
23	Диплом	Разделы 3.1.1 и 3.1.2 поменяны местами	Обязательно
24	Диплом	GPT-4o исправлен на GPT-4o-mini во всех упоминаниях	Обязательно
25	Диплом	Добавлен раздел 2.4 «Разработка отдельных программных модулей»	Обязательно
26	Диплом	Раздел 2.7 переименован в 2.2.6	Обязательно
27	Диплом	Список источников обновлён (добавлены Node.js, Express, Prisma)	Обязательно
28	Демо	Презентация готова (15-20 слайдов)	Обязательно
29	Демо	Live-demo подготовлено (backup-запись на случай сбоев)	Желательно

Заключение: данный план обеспечивает полное соответствие заданию на дипломный проект при сохранении высокого технического уровня реализации. Ключевые архитектурные решения (трёхуровневая архитектура, AI Service Layer с dual provider, безопасное выполнение кода в Web Workers) демонстрируют компетенции в fullstack-разработке, проектировании систем и интеграции ИИ-сервисов.

Документ подготовлен: 06.06.2026